DAY 2 OF 3 · INSTRUCTOR-LED TRAINING

Introduction to Terraform on Google Cloud

Day 2: Reusable configurations and controlled changes

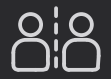
Prepared for Jack Henry & Associates · 09:00–17:00

TODAY'S GUIDING QUESTION

How do we make the deployment reusable while keeping resource identity and changes understandable?

Day outcome: by 17:00 you will have

A documented module



A justified environment-separation approach

A completed drift and import exercise

A reviewed plan for a second region

Day 2 agenda

Time	Session	
09:00–10:30	M07	Functions, collections, and dynamic blocks
10:30–10:45		Break
10:45–12:00	M08	Meta-arguments, lifecycle, and replacement
12:00–13:00		Lunch
13:00–14:10	M09	Modules and reusable interfaces
14:10–14:45	M10	Workspaces and environment separation
14:45–15:00		Break
15:00–16:15	M11	Drift, refresh, and importing resources
16:15–17:00	M12	Applying reuse across regions

BEFORE M07

Retrieval check: where Day 1 left us

01

What does state associate, and where does ours live now?

02

How do inputs, data sources, and outputs connect?

03

Why does a second unchanged plan matter?

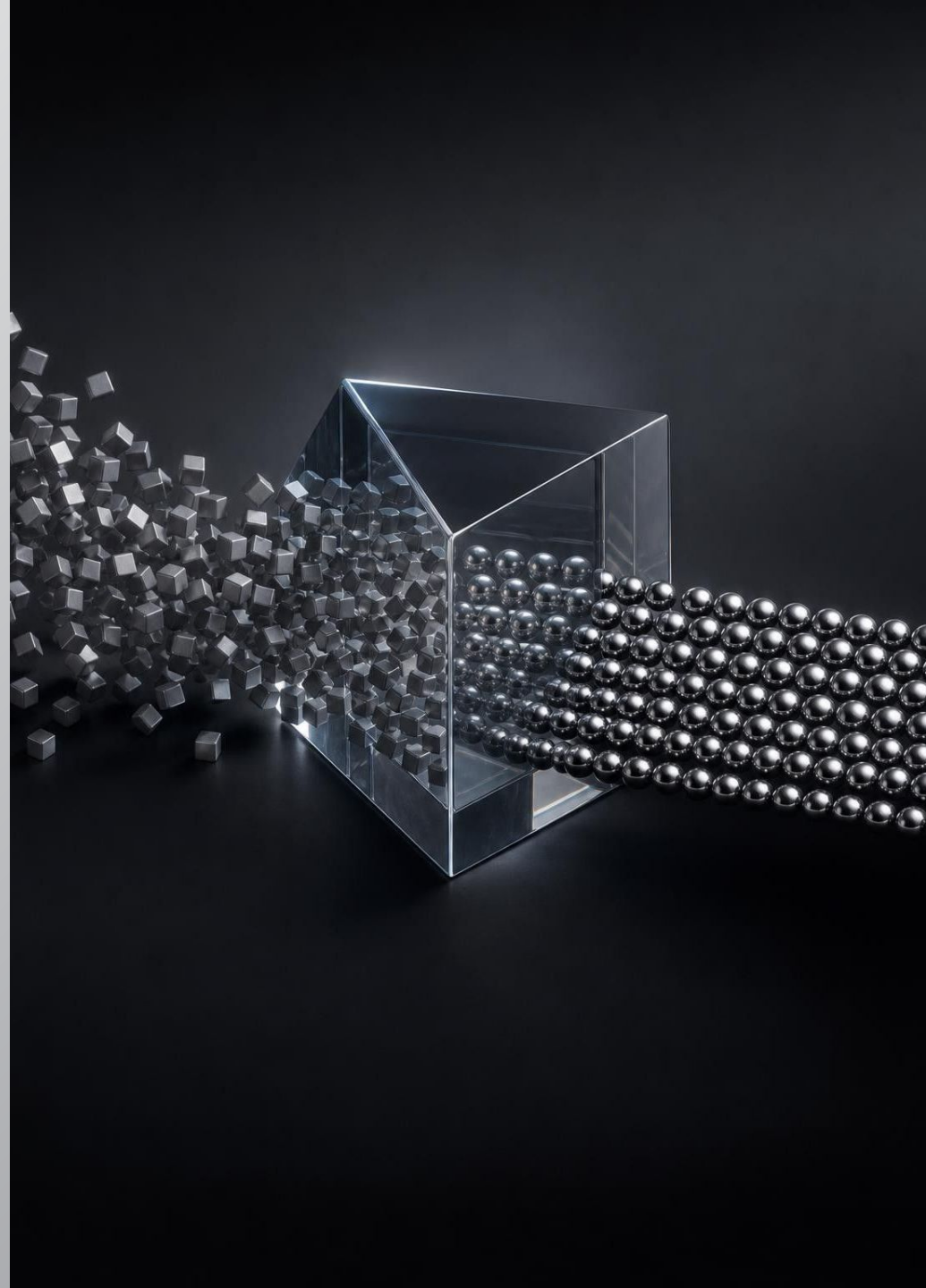
Start from the clean deployment checkpoint before M07.

MODULE · 09:00–10:30

M07

Functions, collections, and dynamic blocks

Shape structured inputs into the values and nested blocks your resources need.



M07 · OUTCOME

Transform inputs, and know what each expression produces

01

Transform

Transform structured inputs into useful values.

02

Distinguish

Distinguish collection expressions, resource iteration, and nested-block generation.

90 minutes

20	20	40	10
Explain	Demo	Practice	Review

Built-in functions you will use today

length

Number of elements.

lookup

Map value with a default.

merge

Combine maps; later keys win.

toset

Convert a list to a set, removing duplicates.

jsonencode

Value to JSON string.

templatefile

Render a template file.

You cannot write functions in HCL, but since Terraform 1.8 providers can publish functions. The Google provider includes provider::google::region_from_zone, name_from_id, project_from_id, and location_from_id.

terraform console: test before you commit

- Interactive evaluation of expressions against the current configuration and state.
- Try functions, for expressions, and variable values safely.
- In L04, predict the result first, then confirm it in the console.

```
$ terraform console
> length(["a", "b"])
2
> merge({a = 1}, {b = 2})
{
  "a" = 1
  "b" = 2
}
> toset(["x", "x"])
toset([
  "x",
])
```

Collection types: list, set, and map

LIST(TYPE)

Ordered

Indexed from 0. Duplicates allowed.

SET(TYPE)

Unique

Unordered. Every value appears once.

MAP(TYPE)

Keyed

String keys to values of one type.

Element types must be consistent within a collection.

Structural types: object and tuple

`object({...})`

Named attributes, each with its own type.

`tuple([...])`

Fixed-length sequence, each position typed.

```
variable "instances" {
  type = map(object({
    machine_type = string
    zone         = string
    labels       = map(string)
  }))
}
```

A map of objects is a common pattern for per-instance settings.

M07 · EXPRESSIONS

for expressions and conditionals

Pattern	Expression	Produces
Transform a list	`[for s in var.list : upper(s)]`	A list
Build a map	`{for k, v in var.map : k => v.zone}`	A map
Filter	`[for s in var.list : s if s != ""]`	A list, filtered by the `if` clause
Conditional	`condition ? true_value : false_value`	One of two results; both should share a type

Dynamic blocks generate repeated nested blocks

- A `dynamic` block iterates over a collection to produce nested blocks inside one resource.
- Example: repeated `allow` blocks in `google_compute_firewall`.
- The iterator is named after the block label by default; `content` defines each generated block.

```
resource "google_compute_firewall" "app" {
  # name, network, and other arguments

  dynamic "allow" {
    for_each = var.allow_rules
    content {
      protocol = allow.value.protocol
      ports    = allow.value.ports
    }
  }
}
```

Limits of dynamic blocks

Cannot

Generate meta-arguments

`dynamic` cannot generate `lifecycle` or other meta-argument blocks.

Avoid

Small, fixed sets

Overuse makes configuration hard to read; prefer literal blocks when the set is small and fixed.

Use

Repetition from data

Use it where a nested block genuinely repeats from data.

Value, nested block, or resource instance?

for expression

Value

Produces a value you can assign, pass, or output.

dynamic block

Nested blocks

Produces nested blocks inside one resource.

resource for_each

Instances

Produces multiple resource instances, each with its own address.

D07 demo and L04 start

D07 · DEMO

Watch

Transform a map of instance settings into labels and outputs, then replace repeated `allow` blocks with one `dynamic` block.

L04 · LAB

Build

Expression experiments, a typed collection, and one useful `dynamic` block.

Show · Evidence

An explicit input type, correct transformed values, and a clear explanation of what each expression produces.

MODULE · 10:45–12:00

M08

Meta-arguments, lifecycle, and replacement

Give instances stable keys, and replace only what you intend to replace.

M08 · OUTCOME

Stable keys and replacements you meant

01

Choose

Choose stable resource-instance keys.

02

Review

Review dependencies and replacement behavior before applying a change.

75 minutes

20	15	30	10
Explain	Demo	Practice	Review

Meta-arguments at a glance

Built into the language and usable on any resource; most also work on `module` blocks.

`count` and `for_each`

Multiple instances.

`depends_on`

Explicit ordering.

`provider`

Select an alternate provider configuration.

`lifecycle`

Customize create and destroy behavior.

count versus for_each

COUNT

Addressed by number

Each instance gets a numeric index.

```
google_compute_instance.app[0]
```

FOR_EACH

Addressed by key

Each instance gets the key it was created from.

```
google_compute_instance.app["web"]
```

`for_each` accepts a map or a set of strings.

Why stable keys matter

count: indexes shift

Removing the first item from a count list shifts every later index. Shifted indexes show up as updates or replacements of the wrong instances.

BEFORE		AFTER REMOVING WEB	RESULT
app[0] · web	→	app[0] · api	changes
app[1] · api	→	app[1] · worker	changes
app[2] · worker	→	removed	destroyed

for_each: one key, one instance

With for_each, removing one key affects only that instance. Every other address stays the same.

BEFORE		AFTER REMOVING WEB	RESULT
app["web"]	→	removed	destroyed
app["api"]	→	app["api"]	unchanged
app["worker"]	→	app["worker"]	unchanged

Dependencies: implicit first, `depends_on` when needed

Implicit by reference

References create implicit dependencies automatically.

What `depends_on` does

It completes all actions on the dependency, including reads, before the dependent object.

Use it selectively

Reserve `depends_on` for dependencies Terraform cannot see in references.

Provider aliases for an alternate context

01

Declare

Add a second `provider` block with `alias = "secondary"`.

02

Select

Choose it per resource with `provider = google.secondary`.

03

Apply the context

Useful when a configuration needs another region or project context.

`lifecycle` settings and their tradeoffs

Setting	Effect
`create_before_destroy`	Create the replacement before destroying the original.
`prevent_destroy`	Any plan that would destroy the resource fails with an error.
`ignore_changes`	Ignore listed attributes after creation. **Can hide real drift.**
`replace_triggered_by`	Replace this resource when a referenced resource or attribute changes.
`precondition` / `postcondition`	Assert assumptions before or after apply; fail the plan with a clear message.

⚠ Learn their effects from small plans; none is general protection against every failure.

Deliberate replacement with `-replace`

1

Preview

`terraform plan -replace=ADDRESS -out=replace.tfplan`

Saves a plan that replaces exactly one resource instance.

2

Review

Confirm the plan shows one `-/+` at the intended address. Check dependent changes and service impact.

3

Perform

`terraform apply replace.tfplan`

Applies the reviewed plan and nothing else.

`terraform taint` is deprecated. Recognize it in older runbooks and use `-replace` instead.

Update in place versus replace

~

Update in place

The provider can change the attribute on the existing object.

-/+

Replace

The attribute forces a new object; the plan marks it as forcing replacement.

Before applying, always cite the resource address and the expected create and destroy actions.

D08 demo and L04 completion

D08 · DEMO

Watch

Compare a collection reorder under count and for_each, then create a reviewed replacement plan for one disposable instance.

L04 · LAB

Build

Give instances stable keys, explain the effect of a collection change, and save a plan that replaces exactly the intended resource.

Apply only the designated lab replacement.

EVIDENCE

Show

The correct address, update versus replacement distinguished, and create and destroy actions justified.

MODULE · 13:00–14:10

M09

Modules and reusable interfaces

Extract a focused child module, document its contract, and keep every existing resource in place.

M09 · OUTCOME

Extract a module without recreating anything

01

Extract

Extract a focused child module and document its interface.

02

Consume

Consume it without unintentionally recreating existing resources.

70 minutes

15	15	35	5
Explain	Demo	Practice	Review

Root and child modules

ROOT MODULE

Working directory

Where you run Terraform. Calls the child with a module block.

`module "app"`

Inputs →

name, zone, subnetwork

← Outputs

instance_name

CHILD MODULE

app_instance

Called with a module block and its own inputs. Returns results only through outputs.

`./modules/app_instance`

Outputs are the module's only interface back to the caller.

Dependencies flow through module inputs and outputs.

A practical module structure

```
modules/app_instance/
├── main.tf
├── variables.tf
├── outputs.tf
├── versions.tf
├── README.md
└── examples/
```

Four core files

`main.tf`, `variables.tf`, `outputs.tf`, and `versions.tf`.

Documented inputs

Descriptions and validation on every input.

Usage examples

An `examples` folder showing usage.

README

A concise README describing inputs, outputs, and purpose.

Module sources

Local path

In this repository

`source = "./modules/app_instance"`

Registry

Published module

Set `source` plus a `version` constraint.

Git

Repository reference

Pin a tag or commit with `?ref=`.

Pin versions explicitly wherever supported.

Keep the module contract narrow

Inputs

Expose only the inputs callers actually need.

Outputs

Return identifiers through outputs rather than requiring callers to reach inside.

Payoff

Narrow contracts are easier to maintain, test, and upgrade.

Calling the module from the root

```
module "app" {
  source     = "./modules/app_instance"
  name       = var.name
  zone       = var.zone
  subnetwork = google_compute_subnetwork.app.id
}

output "instance_name" {
  value = module.app.instance_name
}
```

01

Call

`module "app"` with `source`, then each required input.

02

Read

Use results as `module.app.OUTPUT_NAME`.

03

Initialize

Run `terraform init` after adding or changing a module source.

Moving resources into a module changes their addresses

Before

`google_compute_instance.app`

After

`module.app.google_compute_instance.this`

The default plan

Without guidance, Terraform plans to destroy the old address and create the new one.

Your check

Always inspect the refactor plan before continuing.

The `moved` block preserves identity

```
moved {
  from = google_compute_instance.app
  to   = module.app.google_compute_instance.this
}
```

General form: `moved { from = OLD_ADDRESS  to = NEW_ADDRESS }`

Renamed, not recreated

Terraform checks state for the `from` address and renames it before planning; nothing is destroyed.

Version

Requires Terraform 1.1 or later; `terraform state mv` is the older CLI alternative.

Keep it in place

Removing a `moved` block later is a breaking change for anyone who has not yet applied it.

D09 demo and L05 start

D09 · DEMO

Watch

Extract the application instance pattern into a module and compare the unsafe refactor plan with the corrected address migration.

L05 · LAB

Build

Define inputs and outputs, call the module from the root, use the supplied moved mapping, and add a usage example.

EVIDENCE

Show

A documented interface, usable outputs, and a refactor plan with no unintended replacement or destruction.

MODULE · 14:10–14:45

M10

Workspaces and environment separation

Know which workspace is selected, and choose boundaries that match real access needs.

M10 · OUTCOME

Know your workspace, choose real boundaries

01

Select

Select the intended CLI workspace.

02

Explain

Explain when separate root configurations, backends, projects, and identities are needed.

35 minutes

10	10	10	5
Explain	Demo	Practice	Review

CLI workspace commands

`terraform workspace`

LIST NEW SELECT SHOW DELETE

The default workspace

Every working directory starts with a workspace named `default`, which cannot be deleted.

Scope of commands

Most commands act only on the currently selected workspace.

Our backend

GCS supports multiple workspaces.

The wrong-workspace risk

- Workspaces share one configuration and one backend, each with its own state.
- Applying in the wrong selected workspace changes the wrong environment.
- A plan in a new, empty workspace does not see resources in other workspaces; they still exist.

BEFORE EVERY APPLY

terraform workspace show

Use explicit naming, and confirm the selected workspace before you apply.

CLI workspaces are not access boundaries

HashiCorp: workspaces are not appropriate for deployments requiring separate credentials and access controls.

Terraform documentation on CLI workspaces

Not the same feature

CLI workspaces differ from HCP Terraform workspaces.

Course pattern

Separate development and test roots with separate state locations.

D10 demo and L05 continued

D10 · DEMO

Watch

Switch workspaces in a disposable configuration; selecting an empty state does not remove resources in another workspace.

L05 · LAB

Build

A workspace-selection check and a comparison of two environment layouts.

Choose the layout for a team that needs separate access rights.

EVIDENCE

Show

Identify the active workspace, and justify a boundary using both state and identity.

MODULE · 15:00–16:15

M11

Drift, refresh, and importing resources

Decide what to do when reality and configuration disagree, and bring existing resources under management.

M11 · OUTCOME

Decide on drift, adopt an existing resource

01

Decide

Decide whether to restore infrastructure or accept a remote change.

02

Adopt

Adopt one existing resource into a reviewed configuration.

75 minutes

20	20	30	5
Explain	Demo	Practice	Review

Three drift cases

Approved emergency change

Made outside Terraform under urgency.

Accidental console edit

An unintended change.

Externally created resource

Now needs Terraform ownership.

Two responses to drift

A normal plan refreshes and shows divergence from configuration.

RESTORE

Return to the configured state

Apply the reviewed plan to return infrastructure to what the configuration describes.

ACCEPT

Adopt a legitimate change

Update the configuration to match the change, then confirm a clean plan.

Refresh-only mode

01

`terraform plan -refresh-only`

Shows what changed remotely without proposing infrastructure changes.

02

`terraform apply -refresh-only`

Updates state and outputs after review.

Refresh-only never changes infrastructure or configuration.

`terraform refresh` is deprecated; it is equivalent to `apply -refresh-only -auto-approve` and skips review.

Why auto-applied refresh is risky

1

Misread

With misconfigured credentials, Terraform may conclude all managed objects were deleted.

2

Removed

Auto-approve would then remove them from state with no prompt.

3

Prefer

Use a reviewed `apply -refresh-only`, or rely on the refresh in a normal plan.

Importing with an `import` block

01

Map

Add an `import` block with `to` = the target address and `id` = the provider's resource ID.

02

Generator or describe

Run `terraform plan -generate-config-out=generated.tf` to have Terraform write the resource block for you or write it by hand.

03

Reconcile

Review the import plan, apply, then adjust arguments until the next plan is clean.

```
import {
  to = google_compute_firewall.legacy
  id = "projects/PROJECT/global/firewalls/NAME"
}

resource "google_compute_firewall" "legacy" {
  # arguments matching the existing firewall
}
```

Import blocks support `for_each` for similar resources, `identity` in place of `id` for providers that publish resource identity, and work inside child modules as of Terraform 1.16.

The CLI import route

RECOGNIZE

`terraform import ADDRESS ID`

- Imports one object directly into state.
- Not recorded in configuration.
- Not previewed in a plan.

COURSE ROUTE

`import` **blocks**

The course uses import blocks for reviewable imports: the import lives in configuration and you review its plan before applying.

Reading, accepting, and desiring are separate steps

Read

Read remote values

Refresh during planning.

Accept

Accept state changes

A reviewed `apply -refresh-only`.

Desire

Change desired configuration

Edit `.tf` files and apply a reviewed plan.

D11 demo and L06 lab

D11 · DEMO

Watch

Compare drift responses and import a prepared lab resource.

L06 · PART 1

Diagnose

Diagnose an out-of-band label edit and record the decision.

L06 · PART 2

Import

Import a pre-created resource with a starter block and record the resulting plan.

EVIDENCE

Show

A decision that matches the change request, one correct address, and no unintended changes.

MODULE · 16:15–17:00

M12

Applying reuse across regions

Place the M09 module in a second region without touching the first deployment.

M12 · OUTCOME

A second region without disturbing the first

01

Reuse

Reuse the M09 module for a second regional placement.

02

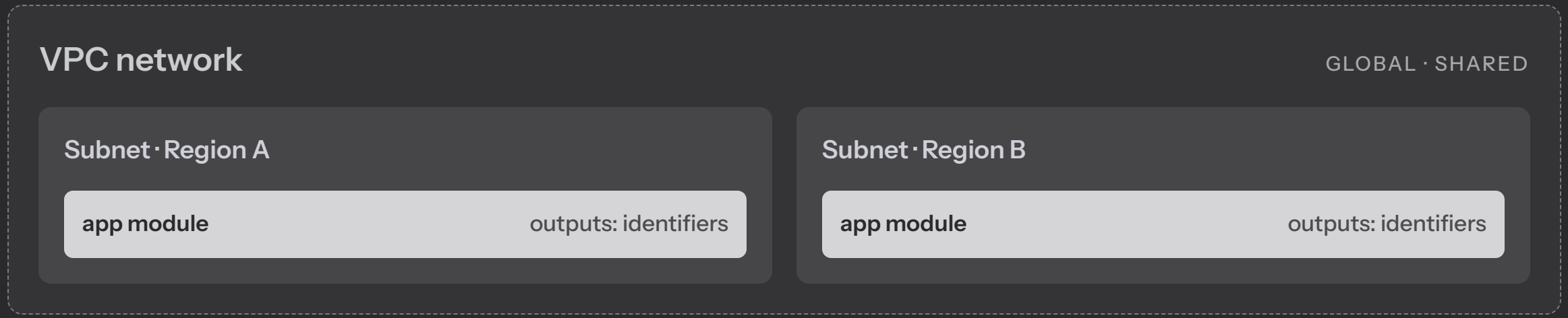
Assess

Assess the proposed change without disturbing the original deployment.

45 minutes

5	5	30	5
Explain	Demo	Practice	Review

Regional versus shared resources



Placement

Region and zone inputs drive placement.

Scope

Subnets are regional; VPC networks are global.

Names

Resource names must stay unique across placements.

Outputs

Module outputs expose identifiers for each region.

A second root with its own state

ORIGINAL ROOT

Left alone

The original deployment's state is never touched.

SECOND ROOT

Prepared for the new region

- Use a prepared second root with a distinct state location and explicit inputs.
- Consume shared-network identifiers through supplied non-secret values or approved lookups.

M12 · SCOPE CHECK

Two regions is not high availability by itself

Production availability also needs

Traffic routing and health checks

Data consistency across regions

Operational design and runbooks

D12 demo and L07 lab

D12 · DEMO

Watch

Inspect a deployed two-region example and its module calls.

L07 · LAB

Build

Add a second-region configuration with the M09 module, validate it, and review its plan.

Second-region apply is an optional extension when quota and time permit.

EVIDENCE

Show

The correct region and state, the original untouched, and meaningful handoff outputs.

DAY 2 · CLOSE

Day 2 wrap-up and retrieval check

1. What will the plan create, update, replace, or destroy, and at which address?
2. How does a value flow through a module to a non-secret output?
3. Which workspace, root, and backend prefix are selected?
4. Which drift or import action fits the change request?

Tomorrow: troubleshooting, state recovery, landing zones, operational security, and Packer.